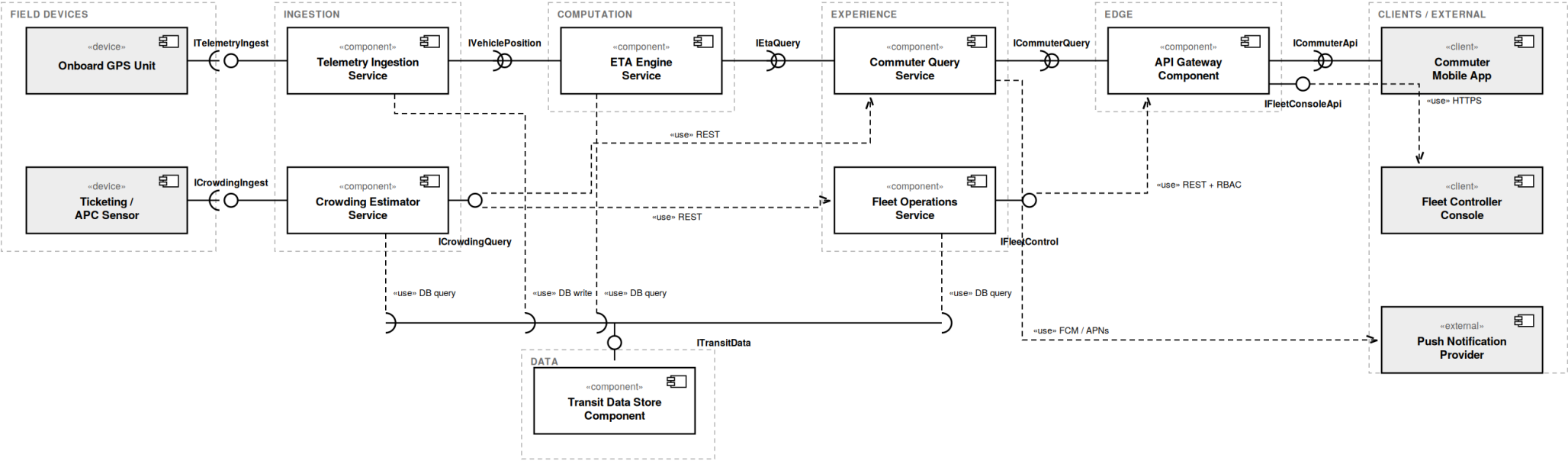


Public Bus Live Tracking & Crowding Estimator — UML Component Diagram

Architecture: Microservices | Problem Statement #24 | SRN PES1UG24CS392 | Section G



| NOTATION |                                                             |
|----------|-------------------------------------------------------------|
|          | Provided interface (ball) — a service the component offers  |
|          | Required interface (socket) — a service the component needs |
|          | «use» dependency — labelled with the protocol / technology  |

| SUMMARY                   |    |
|---------------------------|----|
| System components         | 7  |
| Interfaces                | 10 |
| External actors / systems | 5  |

# Architectural Justification

**Problem Statement #24 — Public Bus Live Tracking & Crowding Estimator** SRN PES1UG24CS392 · Section G · PES University, Dept. of CSE

## Architecture Selection

We chose the **Microservices Architecture** for the **Public Bus Live Tracking & Crowding Estimator System**.

The system is decomposed into seven independently deployable components — API Gateway, Telemetry Ingestion Service, ETA Engine Service, Crowding Estimator Service, Commuter Query Service, Fleet Operations Service and Transit Data Store — communicating over explicit REST and event-stream interfaces rather than in-process calls.

### 1. Architectural Style Analysis

| Style         | Fit against this scenario                                                                                                                                                                                                                                 | Verdict  |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Layered       | Clear separation of concerns, but all layers deploy as one unit. The 200 msg/s telemetry stream and the rush-hour query load would have to be provisioned together, and a fault in the crowding logic sits in the same business layer as ETA computation. | Rejected |
| Client–Server | Simple and consistent, but one centralised server is a single point of failure for a city-wide service and becomes the scalability bottleneck at exactly the peak hour the system exists to serve.                                                        | Rejected |
| Microservices | Each capability scales, fails and deploys on its own. Adds operational complexity and network latency, which is an acceptable cost at this scale.                                                                                                         | Selected |

### 2. Reason One — The workloads have incompatible scaling profiles

The system carries two workloads that grow for unrelated reasons. Telemetry ingestion is a constant, write-heavy stream: 1,000 active vehicles reporting once every 5 seconds is a steady ~200 messages per second that does not vary with how many commuters open the app. Commuter queries are the opposite — near-zero overnight, sharply peaked at rush hour.

Under a Layered or Client–Server design these share one deployable, so the whole application must be provisioned for the sum of both peaks. Microservices lets the ETA Engine scale horizontally at rush hour while Telemetry Ingestion stays at baseline and Fleet Operations — used by a handful of controllers — runs a single instance throughout.

### 3. Reason Two — A sensor failure must not take arrival times down with it

Crowding estimation depends on ticketing and APC sensor hardware fitted to buses, the least reliable input in the system. ETA prediction depends only on the GPS feed. These are independent capabilities, and commuters value the ETA more.

Because the Crowding Estimator is a separate process behind the `ICrowdingQuery` interface, a sensor outage degrades one badge in the UI while live positions and ETAs continue to serve. In a Layered monolith an unhandled fault in the crowding logic can bring down both. This directly supports the fallback specified in Lab 1 (UC-02 alternate flow), where the system degrades a single field rather than failing whole.

### 4. Security Advantage — Role separation enforced at a network boundary

FR-005 lets a Fleet Controller dispatch a relief bus, the only action in the system that changes the physical world, while NFR-002 requires that a Commuter-role token never reaches it.

Here the Fleet Operations Service is a separate deployable whose `IFleetControl` interface is reachable only from the API Gateway, which authenticates and applies role-based access control before routing. The commuter path terminates at the Commuter Query Service and has no network route to dispatch at all. A flaw in the commuter API therefore cannot escalate into control of the fleet: the blast radius is contained by deployment topology, which is stronger than a permission check inside a shared process.

### 5. Performance Benefit — The ETA Engine scales independently

NFR-001 requires a 95th-percentile ETA recomputation under 2 seconds across 1,000 vehicles. As a dedicated service the ETA Engine can be replicated horizontally with route state partitioned by route ID and held in memory, so a recomputation cycle touches local data instead of contending with commuter read traffic on a shared application server. Ingestion is decoupled through the `IVehiclePosition` event stream, so a burst of telemetry queues rather than blocking queries. The p95 target is met by adding ETA Engine instances alone, without redeploying or over-provisioning anything else.